

IMPERIAL

Project MOSAIC

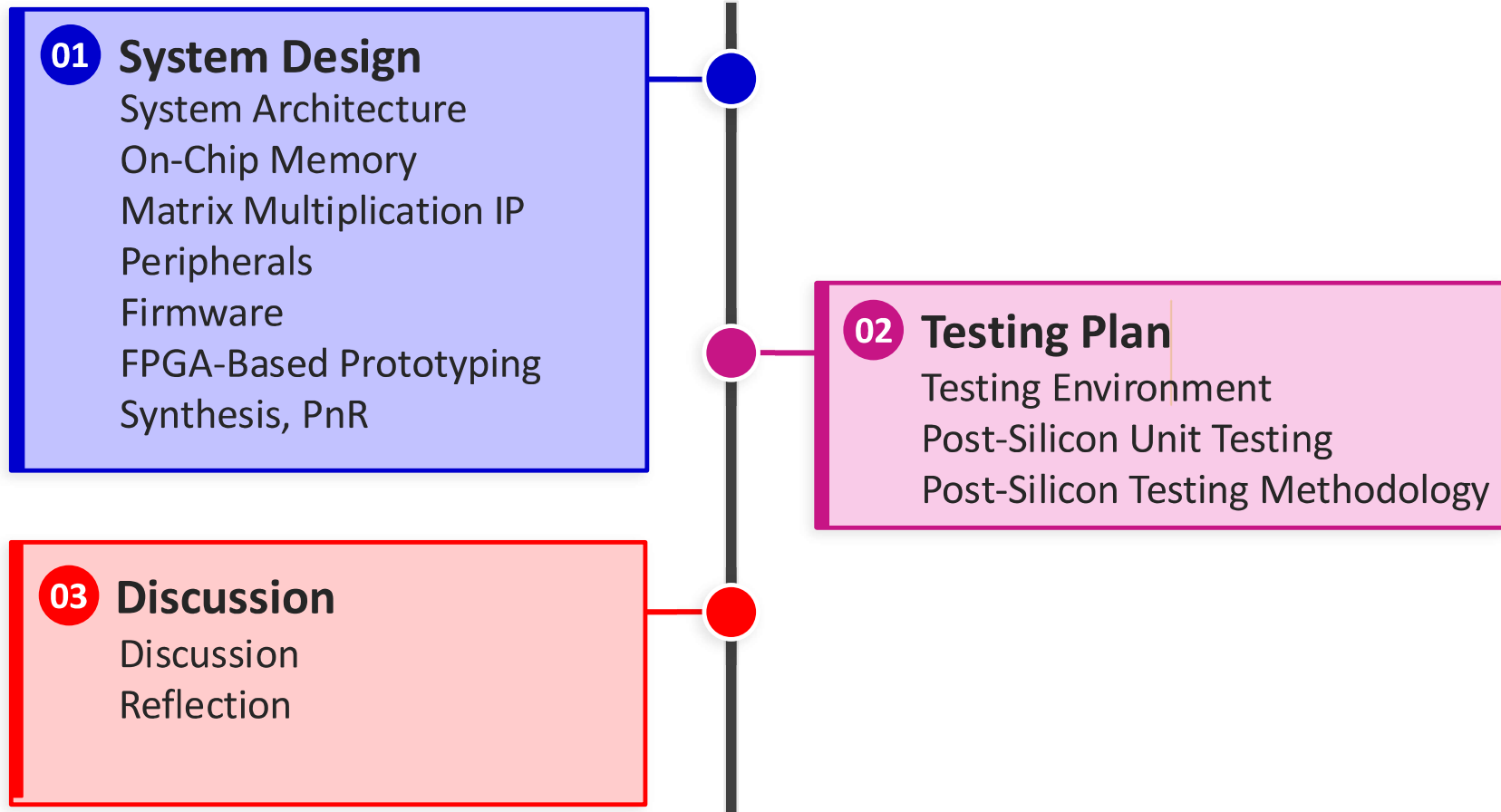
**Matrix-Oriented System-on-Chip with Accelerated
Integrated Compute**

Team Members: Haocheng Fan, Zhihao Jia, Guanxi Lu, Jordon Xu

Supervisors: Prof. Peter Cheung, Peter Liu, Luca Ridolfi, Neeharika Govindu(Apple)

18/06/2026

Table of Contents

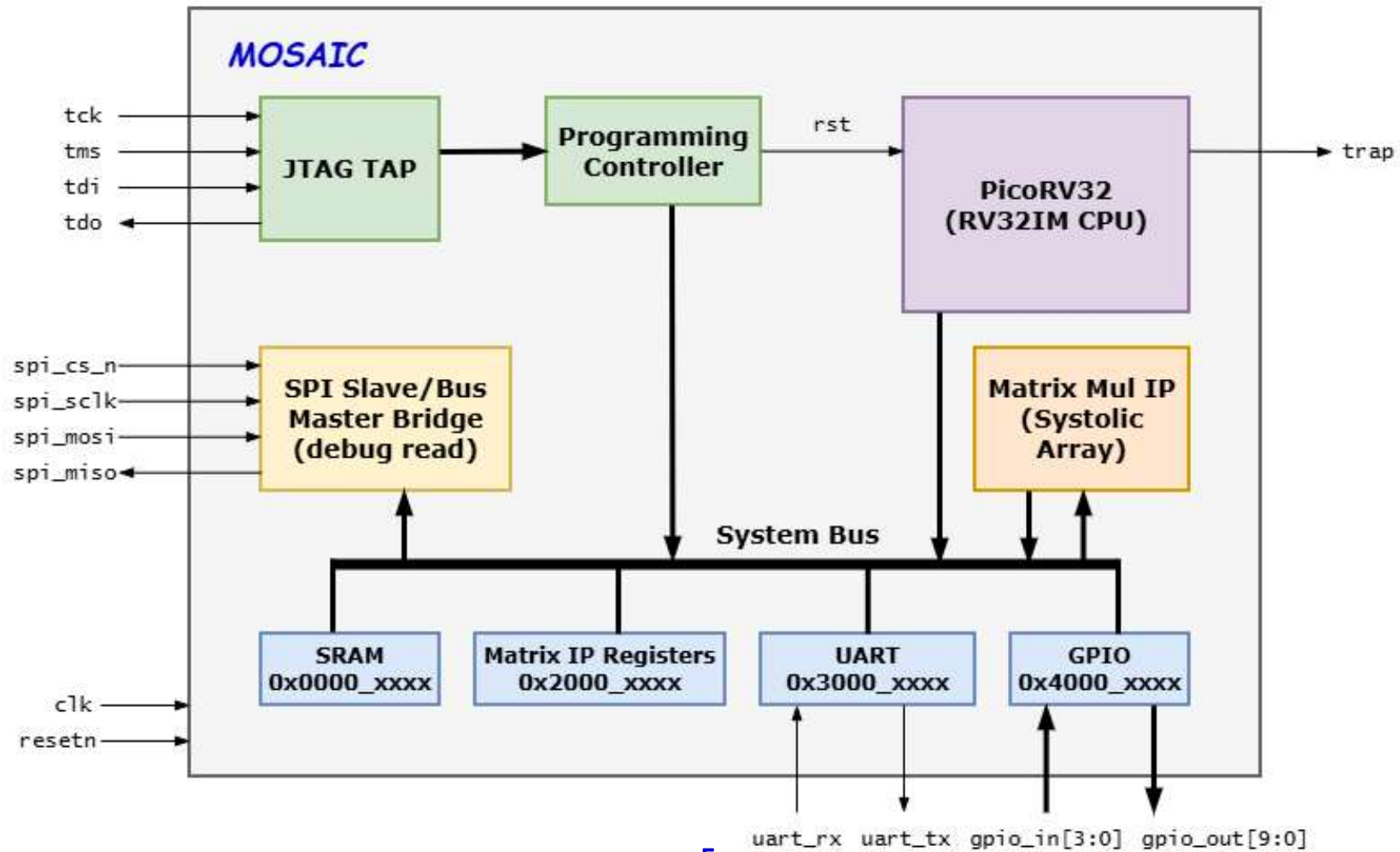


Part 1 – System Design

Motivation

- Edge AI workloads increasingly rely on **on-device preprocessing** and **lightweight linear algebra** to reduce data movement.
- CPU/GPU-centric execution can introduce **high control overhead**, **inefficient fine-grained memory access**, and **poor energy efficiency**.
- Many edge-AI **preprocessing and feature-extraction tasks** reduce to small- to medium-scale structured matrix operations.
- MOSAIC targets these workloads with a tightly integrated **CPU–matrix-accelerator SoC** and unified on-chip memory.

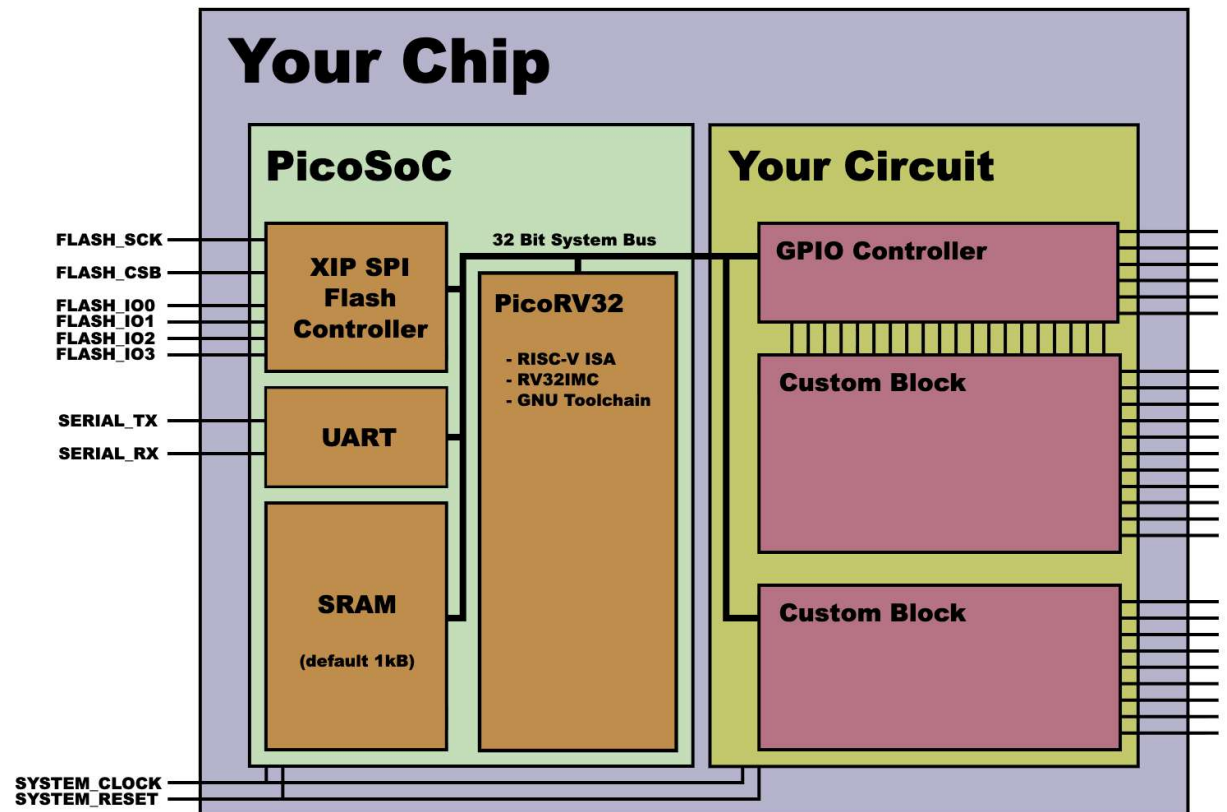
System Architecture



System Architecture – CPU Selection

- **Why PicoRV32?**

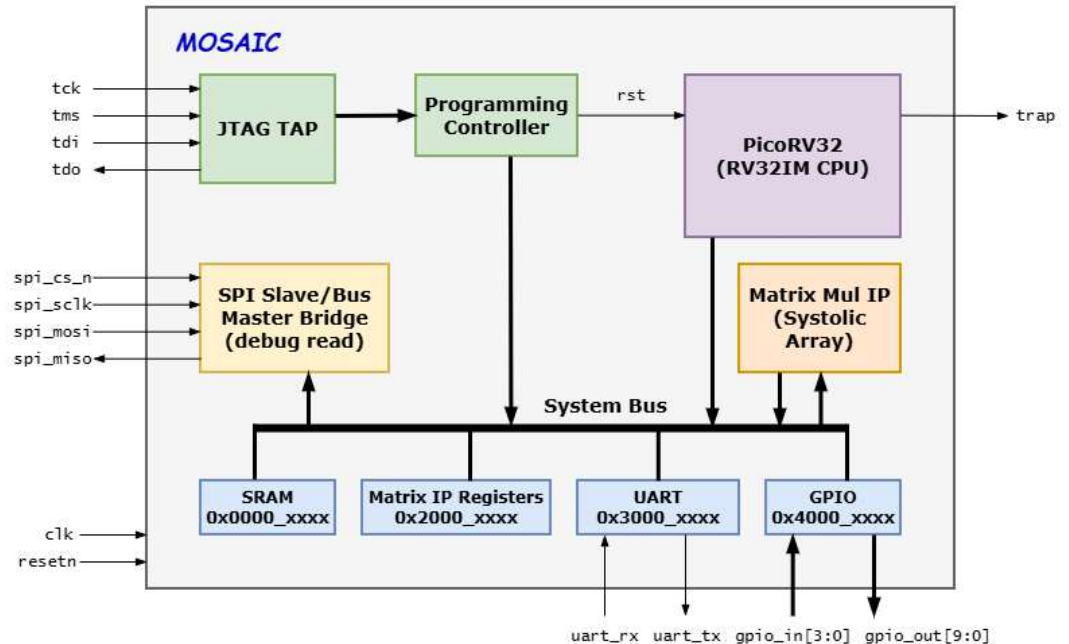
- Small open-source RISC-V core for lightweight system control
- Native memory interface for simple integration
- Direct connectivity to SRAM and memory-mapped peripherals
- Black-box integration to reduce CPU verification risk
- MMIO-based control path for configuring and launching the Matrix IP



<https://github.com/YosysHQ/picorv32/tree/main/picosoc>

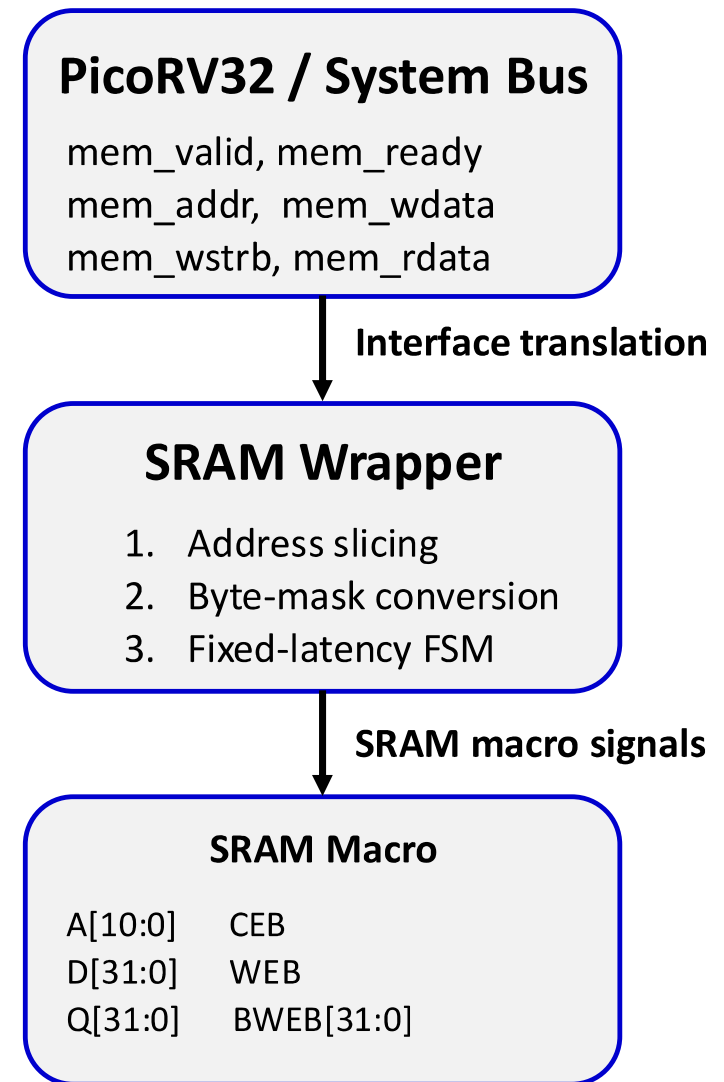
System Architecture – Bus & Memory Map

- **Interconnect** connects all bus masters to shared SRAM and MMIO peripherals.
- **Bus arbiter** selects which master is allowed to use the shared bus.
- **Address decoder** determines which slave should respond based on the memory address.
- **Response path** returns read data and ready signals back to the active master.
- **Matrix IP** is configured through MMIO registers and accesses SRAM for data transfer.

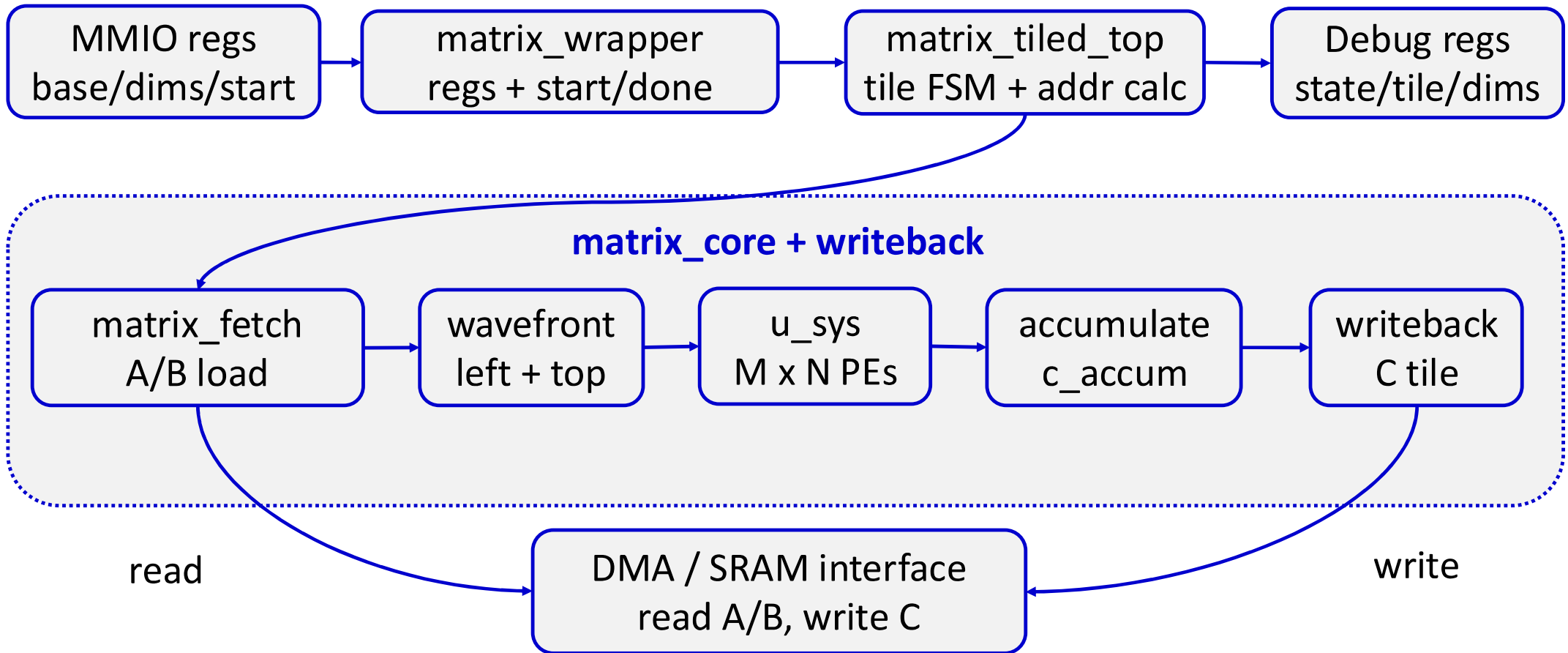


On-Chip Memory

- **Role of On-Chip SRAM**
 - Main on-chip memory
 - Stores firmware, data, and matrix buffers
- **SRAM Macro**
 - 2048 words × 32-bit = 8 KB
 - Word-addressed & Active-low control signals
- **Memory Wrapper**
 - Convert Byte address to word address
 - Convert mem_wstrb to BWEB masp
 - Fixed-latency mem_ready response



Matrix Mult. IP – RTL Architecture



Matrix Mult. IP – Control Flow

Matrix Dims

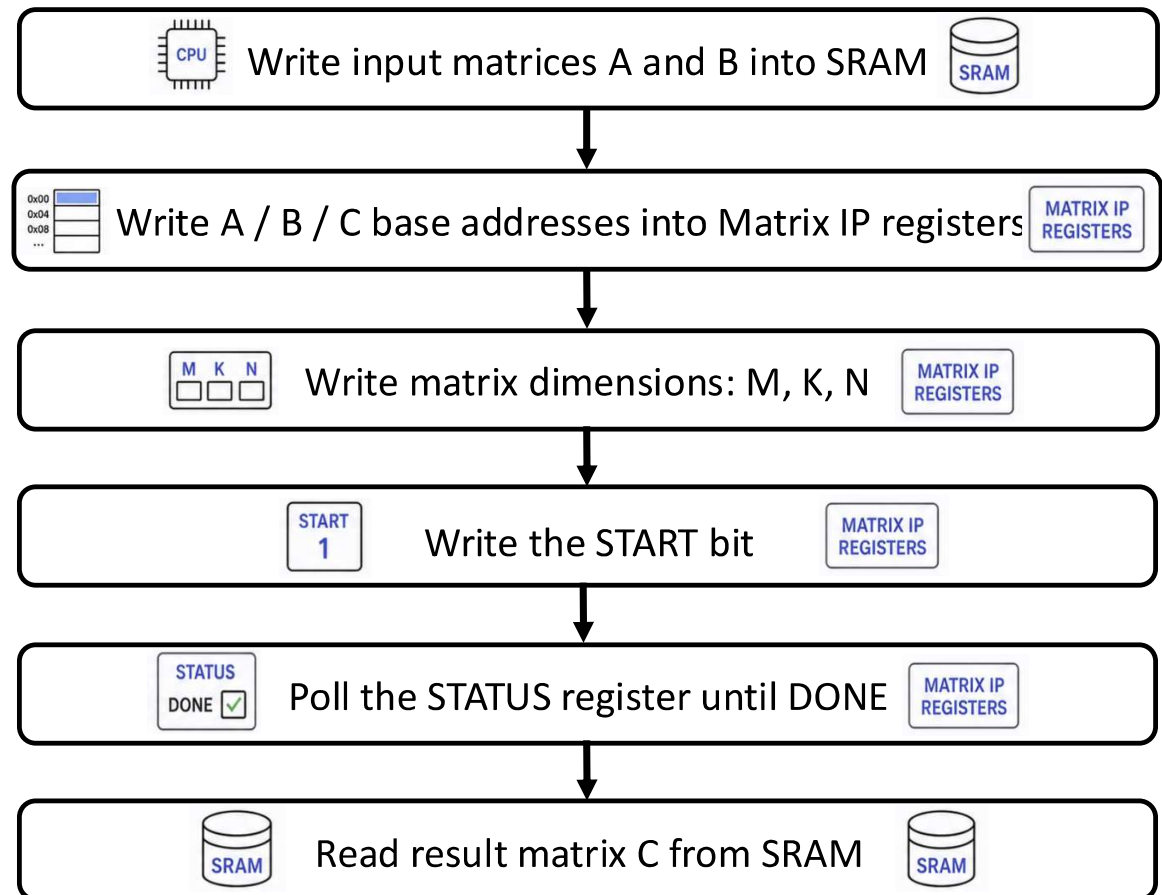
$$A (m \times k) \times B (k \times n) \Rightarrow C (m \times n)$$

Data Format

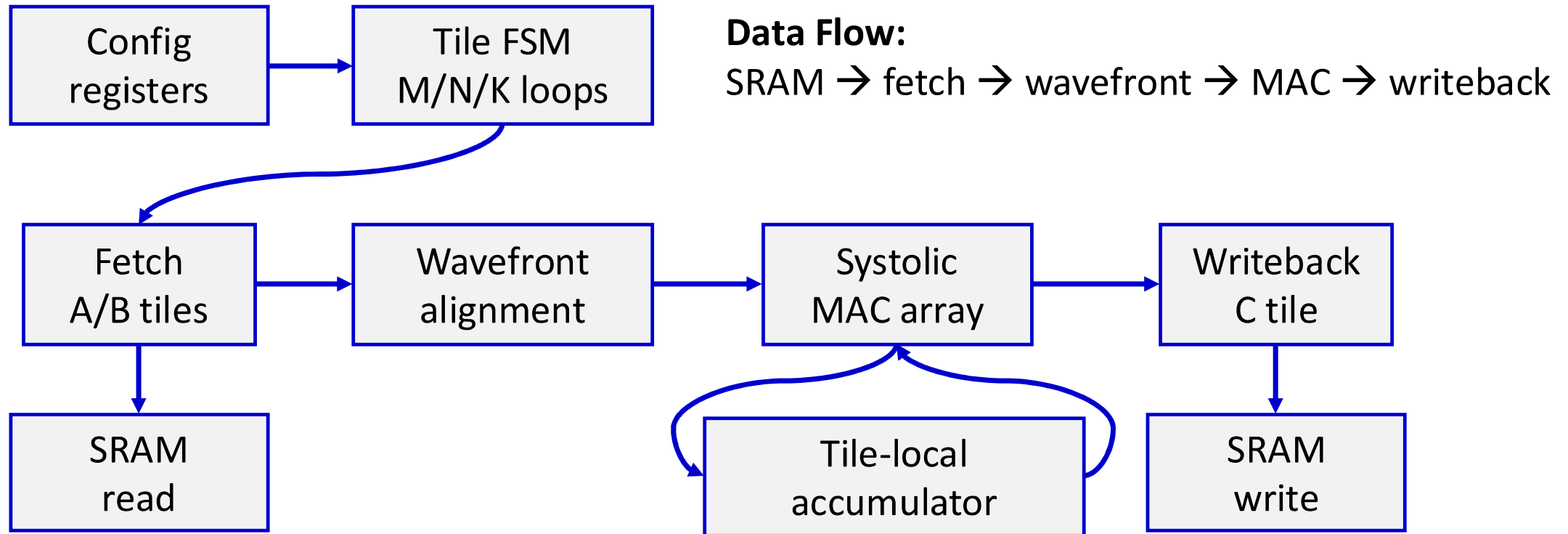
Input: Signed 8-bit
Output: 32-bit

Execution

Start/Done
Autonomous SRAM
Access



Matrix Mult. IP – Data Flow



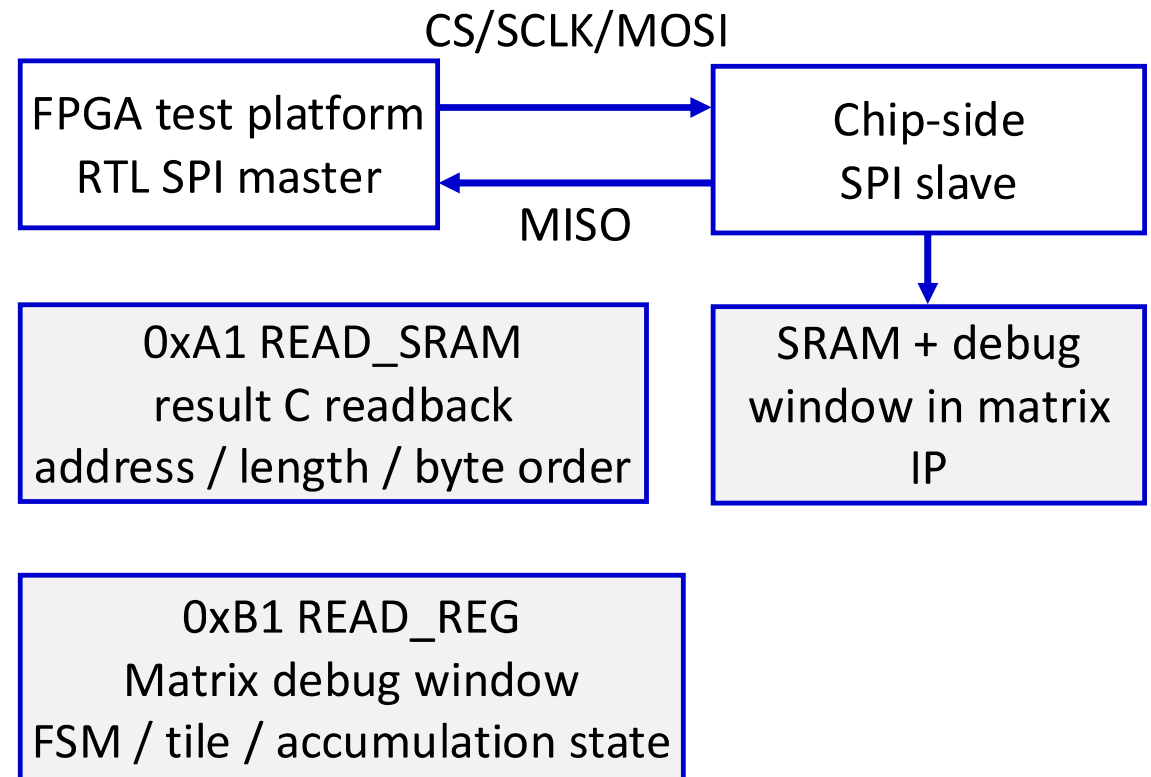
Peripherals – SPI

- **Implementation**

- Chip-side SPI slave
- Mode 0, MSB-first
- 0xA1: READ_SRAM
- 0xB1: READ_REG

- **System Integration**

- External SPI master sends commands
- SPI read path accesses SRAM via bus
- Debug window exposes FSM / tile / accumulation state



Peripherals – JTAG

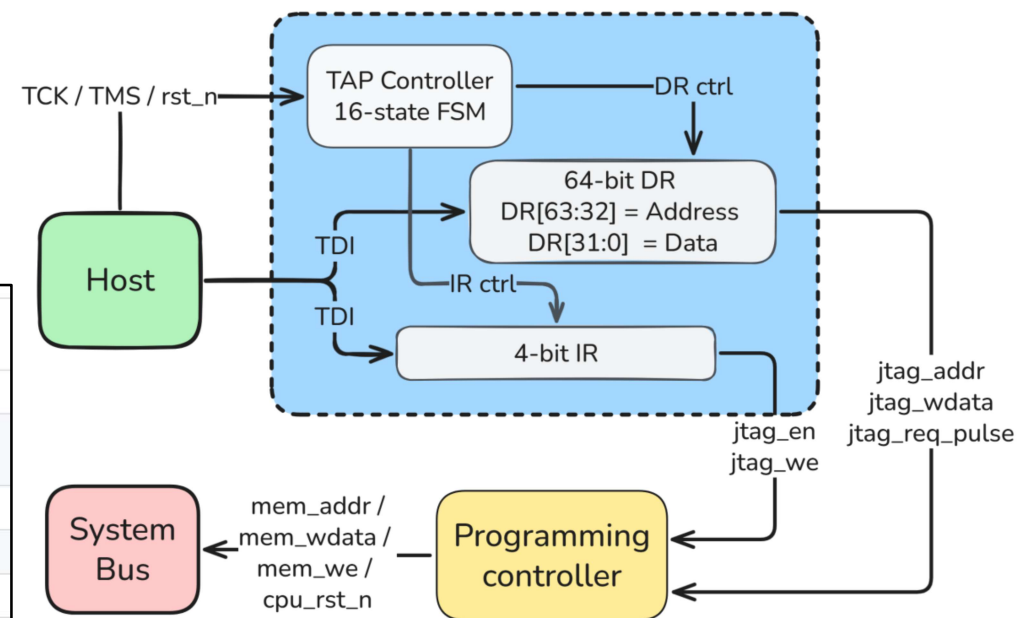
- **Implementation**

- Follows IEEE 1149.1-Compliant TAP Control.
- Controls instruction and data transfers through the standard TAP state machine.

- **System Integration**

- System Bus Master directly accesses SRAM for program loading.
- CPU Start-Up Control releases the bus and resets the CPU after programming.

Instruction	Code (4'b)	Description
IR_NOP	0000	No Operation. Default state.
IR_MEM_CTRL	0001	Request Bus Control. Asserts <code>jtag_en</code> high. This forces the system arbiter to grant the JTAG controller exclusive access to the bus, pausing the CPU.
IR_WRITE	0010	Write Mode. Tells the controller that the content shifted into DR should be written to memory.
IR_READ	0011	Read Mode. Not Used
IR_DONE	0100	Release Bus. De-asserts <code>jtag_en</code> . Returns control of the bus to the CPU and resets the logic.



Peripherals – UART & GPIO

UART

- **Purpose**
 - Serial debug output
 - Test progress and PASS / FAIL messages
 - Useful when internal waveforms are not visible
- **System Interaction**
 - CPU writes characters to UART MMIO register
 - UART sends serial data to the host terminal

GPIO

- **Purpose**
 - Simple external status flags
 - Pin-level observation or LED output
 - Useful for quick bring-up checks
- **System Interaction**
 - CPU writes bit patterns to GPIO MMIO register
 - GPIO drives output pins high or low

Firmware

- **Memory Map**

Region	Base Address	Size	Description
SRAM	0x0000_0000	8 KB	Code, data, stack, matrix buffers
Matrix IP	0x2000_0000	64 B	Hardware accelerator registers
UART	0x3000_0000	16 B	Serial communication
GPIO	0x4000_0000	16 B	General purpose I/O

- **SRAM Partitioning**

Address Range	Size	Usage
0x0000 - 0x17FF	6 KB	.text + .rodata + .data + .bss
0x1800 - 0x18FF	256 B	Matrix A buffer
0x1900 - 0x19FF	256 B	Matrix B buffer
0x1A00 - 0x1BFF	512 B	Matrix C buffer (32-bit outputs)
0x1C00 - 0x1FFF	1 KB	Stack (grows downward from 0x2000)

- **HAL Functions**

Function	Description
delay(count)	Busy-wait delay loop
led_init()	Configure GPIO[9:0] as outputs
led_set(val)	Set LED output value
sw_read()	Read button/switch inputs
uart_init()	Initialize UART
uart_putc(c)	Send single character
uart_puts(s)	Send null-terminated string
uart_put_hex8/32(val)	Print hex value
uart_put_int8/32(val)	Print signed decimal value
write_matrix_i8(addr, data, rows, cols)	Write int8 matrix to SRAM
read_matrix_i32(addr, data, rows, cols)	Read int32 matrix from SRAM
matrix_multiply(addr_a, addr_b, addr_c, m, k, n, shift)	Execute hardware matrix multiply
cpu_matrix_multiply_i32(A, B, C, m, k, n)	CPU reference compute

FPGA-Based Prototyping

- **Why FPGA-based prototyping**
 - Validate the design functionality on real hardware
 - Provide extensive I/O interfaces for testing
 - Identify potential timing and integration issues early
- **Testing Flow**
 - Mapping CPU w/ UART to FPGA
 - Add matrix multiplication IP
 - Verify results through UART output & LED
- **Test Coverage**
 - 10x CPU functionality, 10x GPIO/LED, 5x UART
 - 10x Basic Matrix Mult. , 10x Advanced Matrix Mult.
 - 1x E2E test

```
[1] LED Animation
    Done.

>>> Press KEY[1] to continue...

[2] Matrix Multiplication: CPU vs Hardware IP
-----

Matrix A (Sequential):
[ 1, 2, 3, 4 ]
[ 5, 6, 7, 8 ]
[ 9,10,11,12 ]
[13,14,15,16 ]

Matrix B (Circulant):
[ 2,-1, 0, 1 ]
[ 1, 2,-1, 0 ]
[ 0, 1, 2,-1 ]
[-1, 0, 1, 2 ]

Computing C = A x B using CPU...
CPU Result:
[0, 6, 8, 6 ]
[8,14,16,14 ]
[16,22,24,22 ]
[24,30,32,30 ]

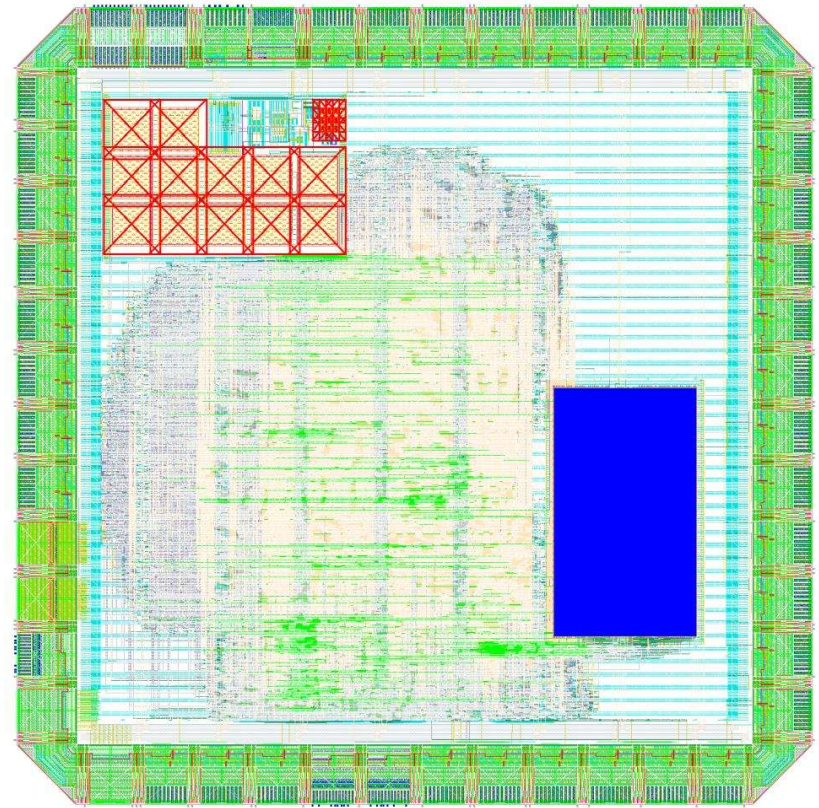
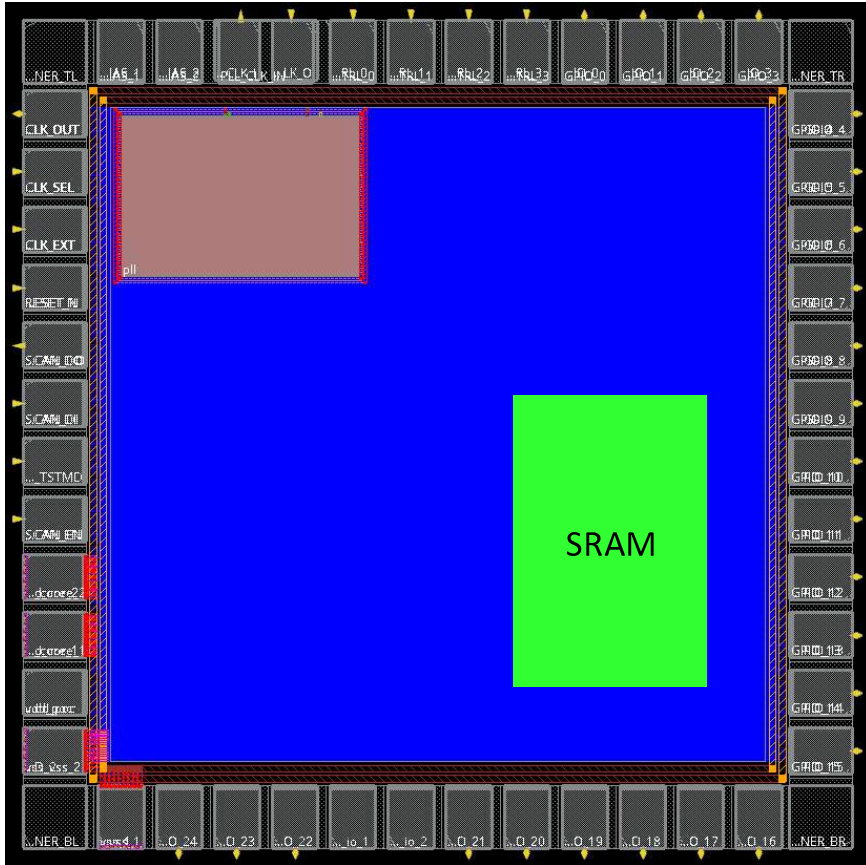
Computing C = A x B using Hardware IP...
Hardware IP Result:
[0, 6, 8, 6 ]
[8,14,16,14 ]
[16,22,24,22 ]
[24,30,32,30 ]

[3] Verification
-----

=====
RESULT: ALL 16 ELEMENTS MATCH!

CPU and Hardware IP produce identical results.
Project MOSAIC verification PASSED!
=====
```


Synthesis, PnR – Floorplan



Synthesis, PnR – Pin Configuration



Pin	soc_top Signal	Description	Direction
gpio[9:0]	gpio_out[9:0]	General purpose output	Output
gpio[13:10]	gpio_in[3:0]	General purpose input	Input
gpio[14]	uart_tx	UART transmit	Output
gpio[15]	uart_rx	UART receive	Input
gpio[16]	tck	JTAG test clock	Input
gpio[17]	tms	JTAG test mode select	Input
gpio[18]	tdi	JTAG test data in	Input
gpio[19]	tdo	JTAG test data out	Output
gpio[20]	spi_cs_n	SPI chip select	Output
gpio[21]	spi_sclk	SPI serial clock	Output
gpio[22]	spi_mosi	SPI master out slave in	Output
gpio[23]	spi_miso	SPI master in slave out	Input
gpio[24]	trap	CPU trap indicator	Output

Part 2 – Testing Plan

Testing Environment

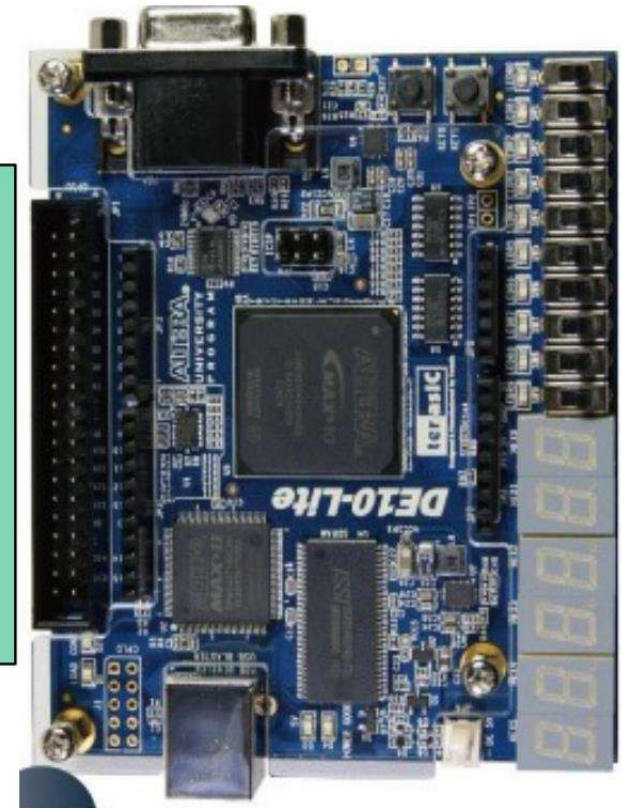
Test Driver Board



Test Board



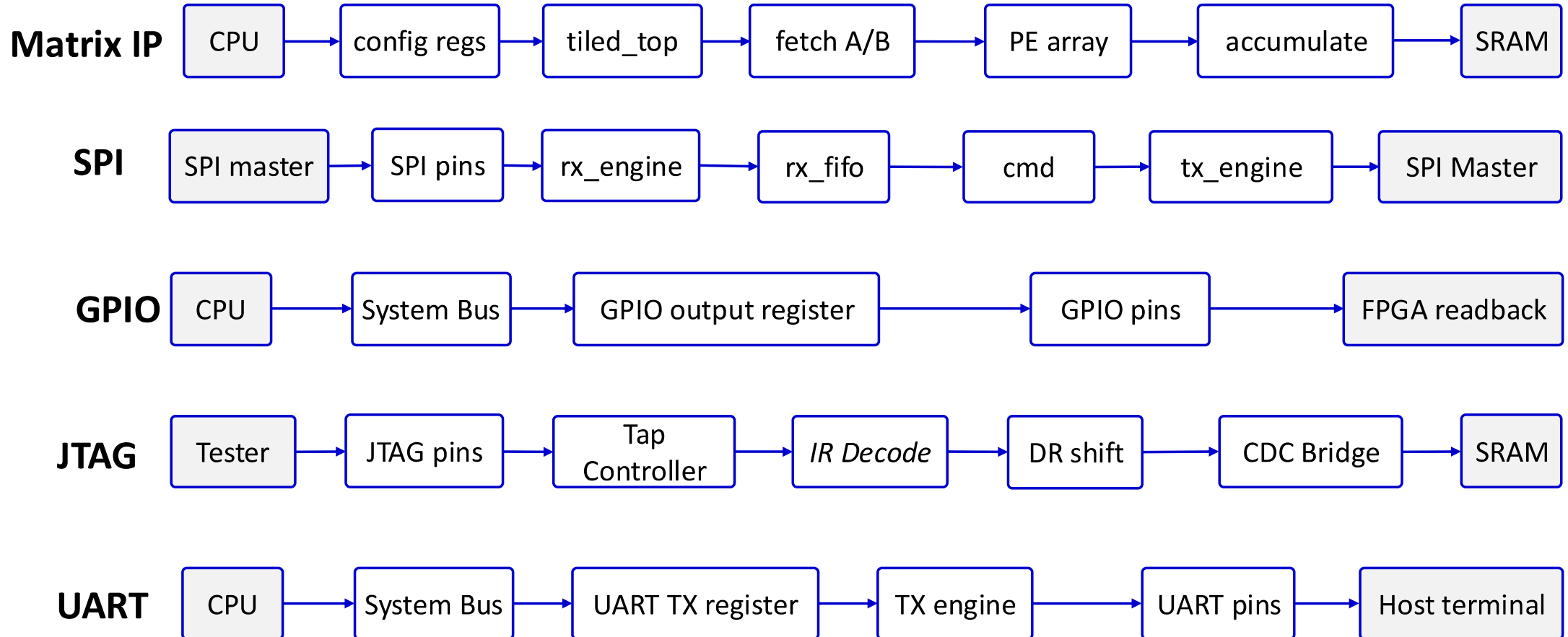
Design Emulation Board



Post-Silicon Testing Methodology

- **System Level Functional Tests**
 - **Boot:** Boot program into SoC SRAM through JTAG
 - **Compute:** Execute Matrix-Multiplication test case
 - **Step :** Read out result through SPI
 - **Verify:** Compare with golden result
- **Scan-Chain Debug Steps**
 - **Shift-in:** inject test vectors via scan-chains.
 - **Execute:** Pulse clock to capture structural/logic responses
 - **Shift-Out:** Extract internal register states for observation.
 - **Compare:** Compare with reference result
- **Internal State Signals:** Prove the target block entered the correct state.
- **Confirm Transfer:** Inspecting signals crossing between distinct IP blocks to catch protocol or timing mismatches.
- **Failure Localization:** Choose checkpoints so that a failure can be isolated to one functional stage.

Post-Silicon Unit Testing



Part 3 – Discussion & Reflection

Discussion

- **Limitations**

- Polling-based accelerator control limits CPU efficiency.
- Int8 matrix multiplication is most effective for dense workloads.
- Further pre-silicon verification (e.g. formal, uvm) is needed.

- **Future Work**

- Add interrupt-based accelerator completion.
- Improve accelerator through overlapping compute and data movement.
- Multiple core run in parallel.
- Perform post-silicon performance and power validation.

Reflection

- **What we learned from this project**
 - **SoC Integration Skill:** how to integrate a RISC-V CPU, on-chip SRAM, a custom matrix accelerator, and peripheral interfaces such as UART, GPIO, JTAG, and SPI into a SoC.
 - **System architecture design skill:** How to design the interactions between the CPU, accelerator, SRAM, firmware, and debug interfaces at the system level.
 - **Verification Skill:** How to verify the design across different stages, including RTL simulation, FPGA prototyping, firmware-based testing, and backend checks.
- **What we would do differently**
 - Start system integration sooner.
 - Make the hardware-software interface more robust.
 - Add more debugging support from the beginning.

Thank You!

With thanks to Prof. Peter Cheung, Peter Liu, Luca Ridolfi, Neeharika Govindu